

Slackbot Implementation Tech Details

This document describes the Slackbot Implementation POC. It is intended for system integrators and project maintainers.

Overview

The Slackbot Integration POC connects a Slackbot with OpenAI's GPT-3.5 turbo and GPT-4 models to enhance accessibility and convenience for users when requesting information about the company.

By leveraging generative AI models, the project enables users to ask questions directly from their preferred Slack channels, receiving real-time personalized and accurate responses.

Architecture

The following architecture diagram shows the components of the Slackbot integration:

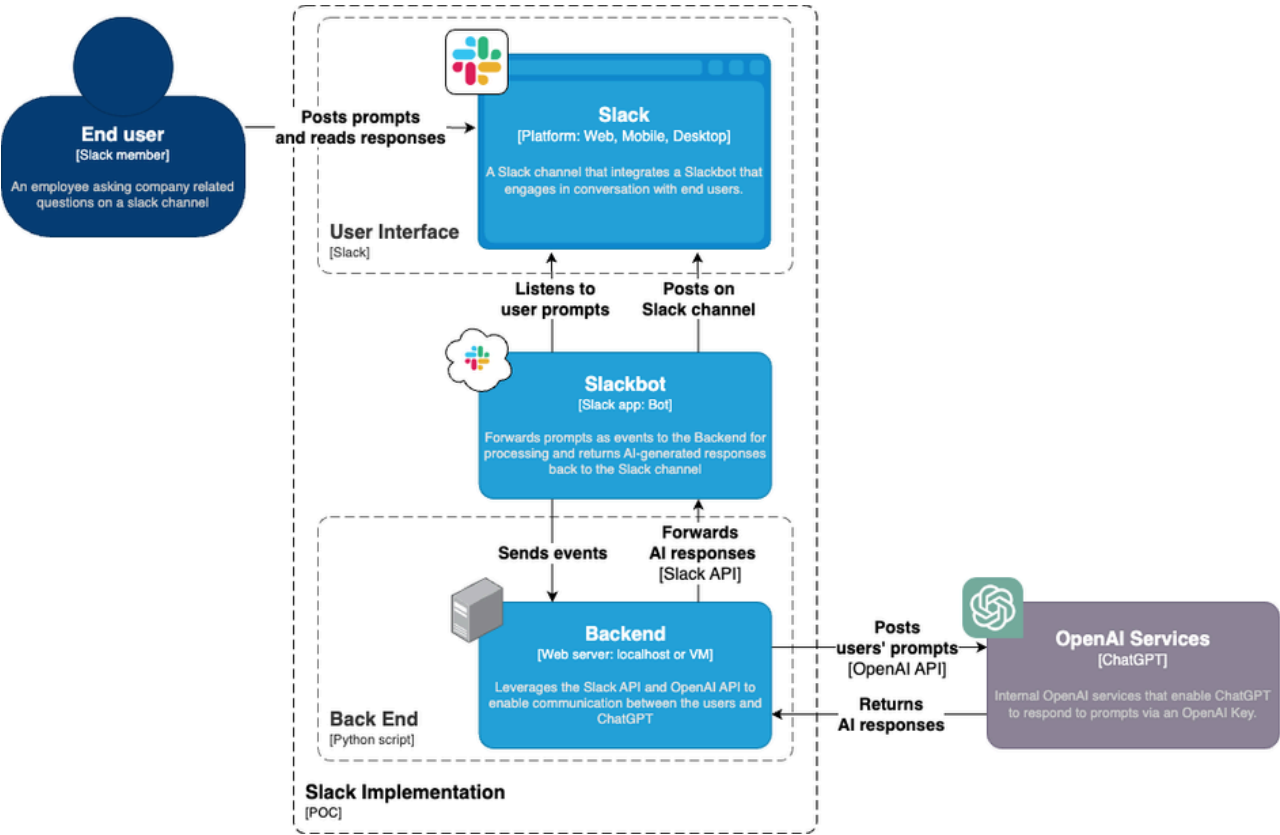


Figure 1. Slackbot Integration Architecture

The architecture comprises the following elements:

Element	Description
User	An employee that is part of a Slack channel
Slack	A cloud-based team collaboration platform that enables real-time communication and file sharing for teams and businesses.

Slackbot	A Slack application that serves as a bot in the Slack channel. Forwards user prompts to the back end and posts the responses back to the Slack channel.
Backend	A web server that leverages the Slack and OpenAI APIs to enable communication between the user and ChatGPT.
OpenAI Services	Infrastructure services that communicate with ChatGPT to formulate a response back to the Backend.

Communication Flow

The following sequence diagram shows the interaction between the elements:

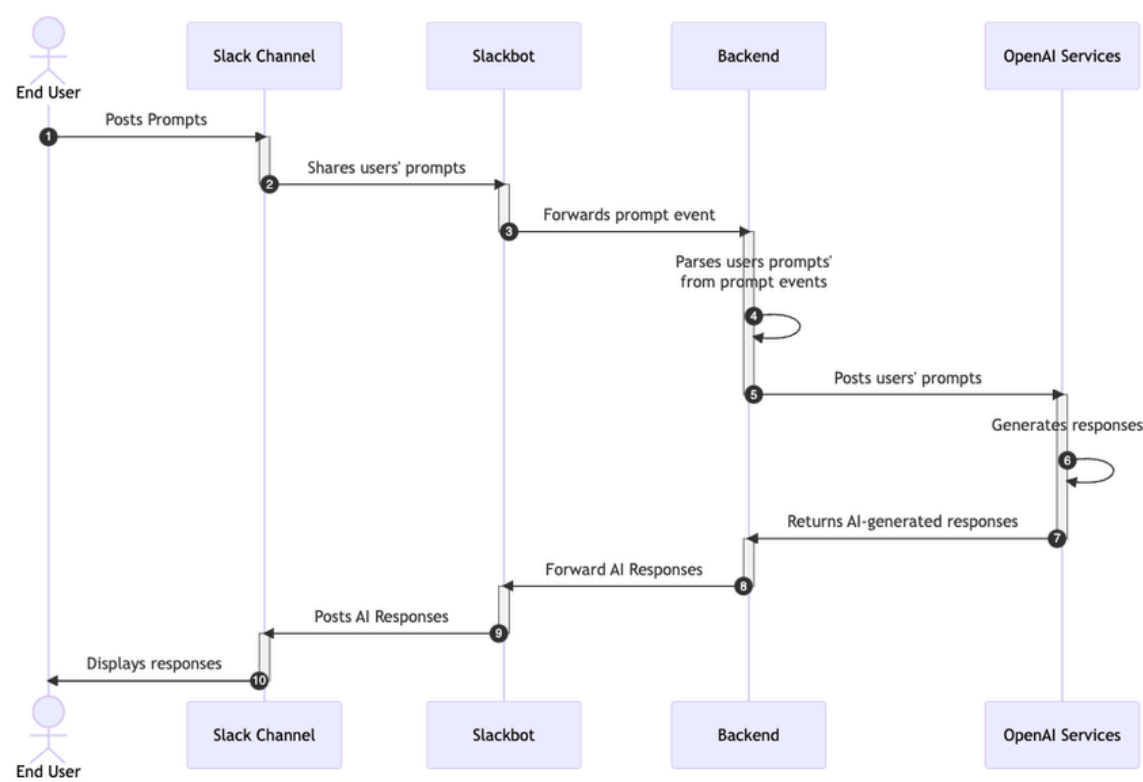


Figure 2. Prompt and response communication

The communication flow of the architecture components is the following:

- 1. An **End User** posts a prompt in the **Slack Channel**.
- 2. The **Slack Channel** shares the user's prompt with the **Slackbot**.
- 3. The **Slackbot** forwards the prompt event to the **Backend** service.
- 4. The **Backend** service parses the user's prompt from the prompt event.

5. The **Backend** service posts the user's prompt to the **OpenAI Services** via the OpenAI API.
6. The **OpenAI Services** generate a response for the prompt
7. The **OpenAI Services** returns the AI-generated response to the **Backend** service.
8. The **Backend** service forwards the AI-generated response to the **Slackbot** via the Slack API.
9. The **Slackbot** posts the AI-generated response in the **Slack Channel**.
10. The **Slack Channel** displays the AI-generated response.

Configuration

Follow the instructions in this section to set up a Slackbot.

Prerequisites

Ensure you have the following dependencies installed before continuing with the Slackbot implementation:

- [Python 3.10 or above](#) must be installed in your terminal
- [pip3](#) must be installed in your terminal
- [ngrok](#) must be installed on your computer.

1. Creating a Slack workspace

Follow the official Slack documentation to [create a Slack workspace](#).

2. Creating the Slackbot

Follow the next steps to create a Slack application:

1. Go to [Slack API > Apps](#).
2. Click the **Create an App** button. The **Create an app** pop-up window opens.
3. Select the **From scratch** option.
4. Under the **App Name** field, name your app.
5. Under **Pick a workspace to develop your app in**, select your Slack workspace.
6. Click the **Create App** button to confirm your application settings. The **Create an App** pop-up window closes and redirects you to the new application dashboard.
7. Click the **Bots** button to configure your Slack application as a Slackbot.
8. Click the **Review Scopes to Add** option.
9. Under **Scopes**, add the `chat:write` scope.
10. Click the **Install App to Workspace** button.
11. Click the **Allow** button.

Successfully installing a Slack application will return its application token. Use the application token in the next section to connect your back-end service to your Workspace application.

3. Connecting your Slackbot to your back-end service

Follow the next steps to connect your Slackbot to a back-end service:

1. Create the Slackbot project directory and install the following dependencies:

```
1 ~$: pip3 install slackclient
2 ~$: pip3 install python-dotenv
```

2. Create a `.env` file to save your application token by adding the following code:

```
1 # .env
2
3 SLACK_TOKEN=Application token
```

ACTION: Replace `Application token` with the string you obtained after completing step 2

3. Create a `bots.py` Python file that will handle the Slackbot integration and add the following code:

```
1 # bots.py
2
3 import slack
4 import os
5 from pathlib import Path
6 from dotenv import load_dotenv
7
8 env_path = Path('.') / '.env'
9
10 load_dotenv(dotenv_path=env_path)
11 client = slack.WebClient(token=os.environ['SLACK_TOKEN'])
```

4. Run the Python script.

Executing the `bots.py` file without errors will validate the connection between your back-end service and Slack workspace application.

4. Posting Hello World from your Slackbot

Follow the next steps to 'Hello World' a test Slack channel:

1. Access your Slack workspace.
2. From the left navigation column, under **Channels**, click the **Add Channel** button.
3. Click the **Create a Channel** button.
4. Name your Slack channel under **Name** and click the **Save** button. Slack creates your new Slack channel and displays it.
5. Enter the following message to your new Slack channel:

```
1 /invite @your_app_name
```

ACTION: Replace `your_app_name` with the name of your Slack app.gave to your Slack app.

6. Add the following line of code at the bottom of your `bots.py` python file:

```
1 # bots.py
2
3 (...)
4
5 client.chat_postMessage(channel='#channel-name', text='Hello World!')
```

ACTION: Replace `channel-name` with the name of your Slack channel.

7. Run the Python script.

Successfully running the Python script will make the Slackbot post *"Hello World"* into your Slack channel.

5. Lift Up your Backend Web Service

Follow the next steps to lift your backend service:

1. Install `Flask` and `Slackeventsapi`

```
1 ~$: pip3 install flask
```

```
2 ~$: pip3 install slackeventsapi
3
```

2. Access [Slack API > Apps](#).
3. Open your Slackbot application dashboard.
4. From the left navigation menu, under **Settings**, click the **Basic Information** item.
5. Find and copy the **Signing Secret** value.
6. Update your `.env` file with the following line

```
1 # .env
2
3 SLACK_TOKEN=Application token
4 SIGNING_SECRET=Application secret #NEWLINE
5
```

ACTION: Replace `Application secret` with the application signing secret string.

7. Update your `bots.py` with the following new lines:

```
1 # bots.py
2
3 import slack
4 import os
5 from pathlib import Path
6 from dotenv import load_dotenv
7 from flask import Flask #NEWLINE
8 from slackeventsapi import SlackEventAdapter #NEWLINE
9
10 env_path = Path('.') / '.env'
11
12 app = Flask(__name__) #NEWLINE
13 slack_event_adapter = SlackEventAdapter(os.environ['SIGNING_SECRET'], '/slack/events', app) #NEWLINE
14
15 load_dotenv(dotenv_path=env_path)
16 client = slack.WebClient(token=os.environ['SLACK_TOKEN'])
17
18 client.chat_postMessage(channel='#channel-name', text='Hello World!')
19
20 if __name__ == "__main__": #NEWLINE
21     app.run(debug=True) #NEWLINE
22
```

8. Run the Python script to launch your localhost web server.

The console will display the localhost URL, by default it's `127.0.0.1:5000`

6. Subscribing Slackbot to Events

Follow the next steps to connect the back-end service with your Slack workspace:

1. Run the **ngrok** application
2. From the **ngrok** terminal, enter the following command:

```
1 ngrok http portNum
2
```

ACTION: Replace `portNum` with the localhost port number, in this case `5000`.

3. Copy the public ID address that **ngrok** provides to route to your localhost web server.

NOTE: This is an example; do not copy this URL.

4. Go to [Slack API > Apps](#).
5. Open your Slackbot application dashboard.
6. From the left navigation menu, under **Features**, click the **Event Subscriptions** item.
7. Toggle **Enable Events** On
8. Under **Request URL**, paste the **ngrok** public ID address appending the `/slack/events` endpoint.

EXAMPLE: `http://ABC123.ngrok.io/slack/events`

9. Under **Subscribe to bot events**, click the **Add Bot User Event** button, and select `message.channels`.
10. Click the **Save Changes** button.
11. From the left navigation menu, under **Features**, click the `OAuth & Permissions` item,
12. Click the **Reinstall App** button.
13. Click the **Allow** button to confirm.
14. Update your `bots.py` with the following new lines:

```
1 # bots.py
2
3 import slack
4 import os
5 from pathlib import Path
6 from dotenv import load_dotenv
7 from flask import Flask
8 from slackeventsapi import SlackEventAdapter
9
10 env_path = Path('.') / '.env'
11
12 app = Flask(__name__)
13 slack_event_adapter = SlackEventAdapter(os.environ['SIGNING_SECRET'], '/slack/events', app)
14
15 load_dotenv(dotenv_path=env_path)
16 client = slack.WebClient(token=os.environ['SLACK_TOKEN'])
17
18 client.chat_postMessage(channel='#channel-name', text='Hello World!') #REMOVE THIS LINE
19
20 # -- COPY THIS BLOCK --
21 BOT_ID = client.api_call('auth.test')['user_id']
22
23 @slack_event_adapter.on('message')
24 def message(payload):
25     event = payload.get('event', {})
26     channel_id = event.get('channel')
27     user_id = event.get('user')
28     text = event.get('text')
29
30     if BOT_ID != user_id:
31         client.chat_postMessage(channel=channel_id, text=text)
32 # -- END OF BLOCK ---
33
34 if __name__ == "__name__":
35     app.run(debug=True)
36
```

ACTION: Remove line 18 and copy the code block from lines 20-32

15. Run the `bots.py` Python script.
16. Create a test post in the Slack channel associated with your Slackbot.

Successfully implementing these steps will allow your Slackbot to reply to your user messages.